

Analysis: Bacterial Tactics

Test set 1

On a player's turn, the grid will be in some *state*, according to whether any bacteria have already been placed, and how they have spread. We can determine whether a state is *losing* via the following recursive definition:

- the player has no moves because there are no empty cells
- any of the player's moves would either cause a mutation, or give the opponent a *winning* state.
 - A winning state is a state that has at least one *winning move* — that is, a move that would leave the other player in a losing state.

Observe that if a state is not losing, it must be winning, since it has at least one move that does not cause a mutation and does not give the opponent a winning state.

To find the number of winning opening moves (if any) for Becca, we can check each move to see whether it is a winning move. Of course, to do this, we have to investigate the resulting state recursively per the above definition. However, since there are up to two moves per empty cell per state, the naive implementation that recursively counts the number of winning moves for each state may not be fast enough to handle even the 4×4 grids in test set 1, so we should optimize it.

Notice that whether a state is winning or losing does not depend on who the player is or on any previous moves. Since the same state may come up multiple times, we should consider memoizing our findings about each state to use in the future. It may be daunting that the number of possible states is intractably large. However, for any given case, there can be at most 16 initially empty cells, each of which can be either filled in by bacteria or not. (After a colony has been placed and has spread, it no longer matters what type it was.) So, we can put an upper bound of 2^{16} on the number of states per case. In practice, there will be even fewer because not all states are reachable.

Moreover, we can save some time by not computing the exact number of winning moves for every state we examine. We only care about this value for an initial state; for every other state, it suffices to determine whether it is winning or losing. If we are investigating a non-initial state's moves and we find a winning move, we can declare the state to be winning, and stop. This optimization alone may be enough to solve test set 1.

Test set 2

When a player makes a legal move, the bacteria spread across the entire width or length of the row or column, up until the line of bacteria reaches the edge of the grid or a cell that is already infected. Therefore, each move creates up to two subproblems that are independent in the sense that a move in one subproblem does not affect the state of the other.

Each subproblem can be expressed as a rectangle contained within the full grid. There are therefore at most $O(R^2C^2)$ subproblems. How can we use the results of these subproblems to determine the overall winner of the game?

The goal of the game is to force the opponent into a situation in which there is no move they can make that leads them down a path to victory. The game is *impartial*: both players have access to

the same set of moves. It is therefore apt to draw a comparison between Bacterial Tactics and the ancient game [Nim](#), an impartial game with similar types of decisions. The mathematics of Nim are well-studied. A discovery particularly useful to us is the [Sprague-Grundy Theorem](#), which says that any impartial game can be mapped to a state of Nim. Every state in Nim corresponds to a non-negative *Grundy number*, or [nimber](#), where any nonzero nimber indicates a winnable game.

According to nimber addition, the nimber of a game state after we place a colony is equal to the XOR of the two subproblems. The nimber of a game state before we place a colony is the [minimum excludant](#), or *MEX*, of the set of possible nimbers after placing colonies. We can therefore solve Bacterial Tactics recursively using the following pseudocode:

```
let solve(state) be a function:
  let s = Å~
  for each legal colony placement:
    add [solve(first subproblem) XOR solve(second subproblem)] to s
  return MEX(s)
```

Given this general framework, we can now optimize our implementation.

First, as in test set 1, we can memoize the game states, which are now defined using rectangles of various sizes within the original grid. Note that it is not possible to have bacteria from previous moves in a subproblem, because we always cut the rectangle along the row or column of cells infected by a colony placement. We may also want to pre-compute the nimbers of all sizes of an empty rectangle (no radioactive cells), which is information that can be shared across all test cases.

Second, observe that if it is legal to place a V colony in a cell, then it is also legal to place a V colony in any cell in that column, and similarly for H colonies in a row, within the boundary of the current subproblem's rectangle. We therefore need to check only the rows and columns for legal colony placements, not each individual cell.

Third, we can construct a data structure that allows us to determine whether a colony placement is legal for any row or column in a given rectangle in $O(1)$ time, allowing us to evaluate any game state in $O(R+C)$ operations. For each row and column in the full grid, create an array. Check the cells in the row or column in ascending order, appending the 1-indexed position of the most recently seen radioactive cell, or 0 if a radioactive cell has not been encountered yet. For example, for the row . # . . #, the array would be [0, 2, 2, 2, 5]. Suppose we have a rectangle that includes the third and fourth cells of that row. The fourth entry of the array is a 2. Since cell 2 is not in our rectangle (we have cells 3 and 4 only), we can conclude that it is safe to place an H colony in this row of our rectangle. This data structure can be pre-computed for each test case in $O(RC)$ time.

To summarize, there are $O(R^2C^2)$ subproblems, and each subproblem takes $O(R+C)$ operations. If we let N be $\max(R, C)$, this leads to $O(N^5)$ total time complexity, sufficient for test set 2. Less efficient solutions might still pass, depending on their implementations.

Analysis: Power Arrangers

Test set 1

We can look at up to 475 figures in Test set 1, but there are 595 figures, so we cannot check them all individually. We can observe that when we investigate any particular set, we only need to look at any four of the figures (say, the first four), since we can infer the identity of the fifth. For example, if the first four figures in a set are D, E, A, and B, then we already know that the last figure in that set is C. This reduces our required number of guesses to $4 \times 119 = 476$. But that is still one more guess than we are allowed!

Observe that once we have determined the orders in all but one of our sets, there are only two sets remaining: the one we have not inspected, and the one we are missing. These two sets must have different figures for at least two indexes, and we can look at any such index to know which set we have. For example, if we know the last two possible sets are CADEB and CDEAB, we can look at the second index; if we see an A, we have the first set, and if we see a D, we have the second set. So, by the time we get to the 119th set on our shelf, we only need to look at one of its figures (but which one to look at will depend on what we learned earlier). After that, we will know which 119 sets we have, and therefore we will also know which set we lack.

Combining these insights, we can guarantee that we will use at most $4 \times 118 + 1 = 473$ guesses.

Test set 2

In Test set 2, we only have 150 guesses, which is not much more than one guess per set! How can we possibly succeed?

We need to take further advantage of the fact that we are missing only one of the possible sets / permutations. We can start by checking the first figure in each set, using 119 guesses. When we do this, we will find that four out of five of the letters occur 24 times each, and the other one occurs only 23 times. That letter must be the one on the first figure in our missing set!

Now we can restrict our search to only our 23 sets that begin with that letter, and look at the second figure in each of those; this uses 23 guesses. One of those letters will appear only 5 times compared to 6 for the others; then we can narrow our search to those 5 out of the 23 sets, and use 5 guesses to look at the third figures of those. We will find one unique third letter, and then we only need to check the fourth figure in that set to learn what the other, missing set is. This uses a total of $(5! - 1) + (4! - 1) + (3! - 1) + (2! - 1) = 119 + 23 + 5 + 1 = 148$ guesses, which fits within the limit of 150.

Analysis: Robot Programming Strategy

With A competitors, there are $A!$ possible initial setups for the tournament bracket, and A is itself exponential in the number of rounds of the tournament. A factorial of an exponential is terrifying enough, and we haven't even started the tournament yet!

Fortunately, we can ignore almost everything about the structure of the tournament. For any opponent, there is at least one possible initial setup in which we will play our first match against that opponent. Since we have to guarantee that we will win the tournament regardless of the initial setup, we must be able to defeat *every* opponent. We cannot tie an opponent, since that coin flip might not come up in our favor!

So all we have to do is find a program that beats every opponent's program. To check a program, we can just check it against each opponent, without worrying about the tournament setup.

Test set 1

In test set 1, there are at most 7 opponents, and their programs can be at most 5 characters long. Our program can be longer than that if need be, but how long does it need to be? We can observe that an optimal winning program should never waste a turn by tying with all remaining opponents, since then it could have instead chosen the move that would have beaten all of them; therefore, it should eliminate at least one opponent per move. So in test set 1, if there is a winning program, it is at most 7 moves long. We can comfortably check all $3^7 + 3^6 \dots + 3$ possible programs of length up to 7.

When simulating a match, we cannot wait around for a googol of moves; by the same argument above, an optimal winning program should take no more than 7 moves to defeat all opponents, so we only need to simulate at most 7 moves. If we are still tied with the opponent at that point, we can safely give up on that program.

If we find that no program of length at most 7 beats every opponent's program, then we can declare that the case is IMPOSSIBLE.

Test set 2

In test set 2, there can be up to 255 opponents, and we cannot generate and check all programs of length up to 255. We must find a way to construct a winning program if it exists.

Let's imagine playing against all opponents at once. How do we choose our program's first move? We must win or tie against every opponent, so we will consider the set of their first moves. If it includes all three possible moves, we are doomed; no matter what we pick, at least one opponent will defeat us. If it includes only one move (e.g. every opponent starts with R), then we can pick the move that defeats that move (in this case, P) and instantly defeat everyone. Otherwise, the set includes two of the possible moves, and we should pick the move that ties one and beats the other. For example, if the opponents all lead off with S or P , we should pick S .

Eliminating any defeated opponents and proceeding to the next move of this combined "match", we can apply the same strategy, but considering the set of remaining opponents' second moves (looping through their programs as needed), and so on. We will eliminate at least one opponent with each move, so after A moves, we will either have our winning program or know that the

case is IMPOSSIBLE. Notice that this limit holds regardless of the lengths of the opponents' programs.